

3 Problem formulation

3.1 Problem setting and assumptions

We consider a multi-story warehouse in which a fleet of autonomous mobile robots (AMRs) transports orders between source and destination floors. To serve an order, an AMR travels to the source floor, collects the load, and delivers it to the destination floor. A set of elevators shared by all AMRs provides the only means of changing floors, so the elevators are the vertical resource for which the AMRs compete. Orders released together form a wave, and their source floors, destination floors, and ready times determine the elevator requests generated when the wave is executed. Fig. 3.1(a) illustrates this setting.

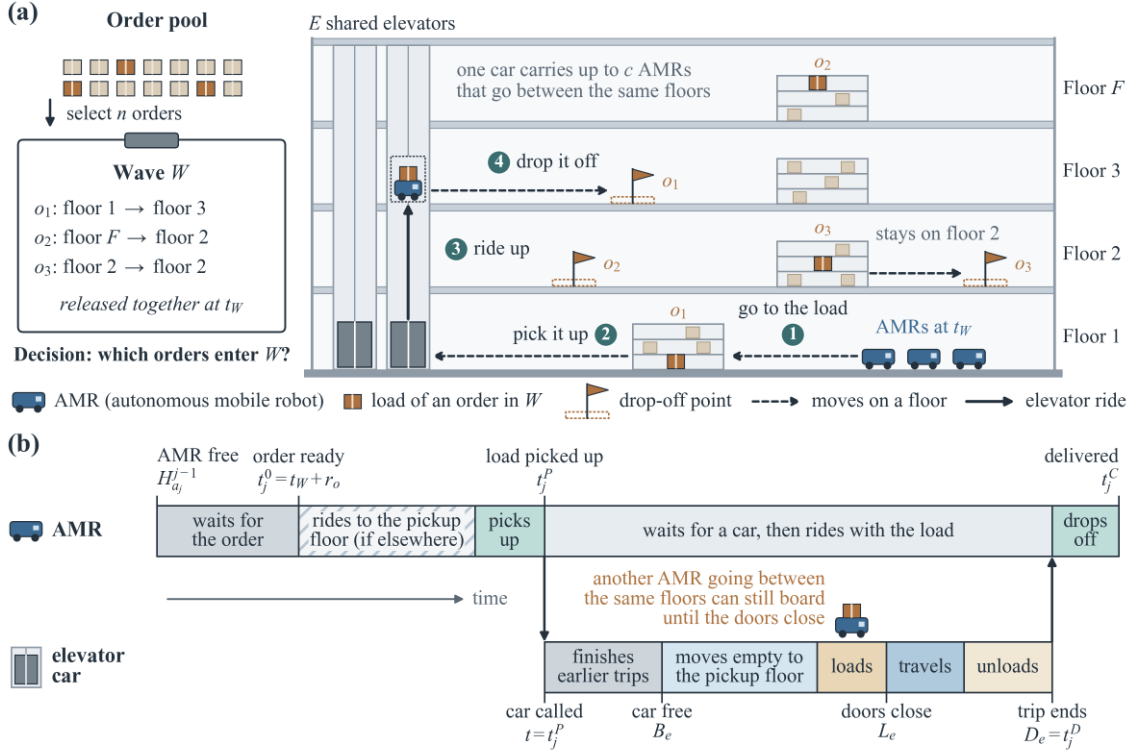


Fig. 3.1. Problem setting and serial reservation of one order. (a) A multi-story warehouse in which an AMR fleet shares E elevators. The decision is which n orders from the order pool form the wave W released at t_W . For each order, an AMR carries a load from its source floor to its destination floor. AMRs change floors only by elevator trips, and a car carries up to c AMRs traveling between the same pair of floors. All AMRs and cars start on a common floor f^0 , drawn as floor 1.

Steps 1 to 4 trace order o_1 ; order o_3 stays on one floor, so its load needs no elevator trip. (b) Reservation of one order under the true co-occupancy batching evaluator M_2 (Section 3.5.1). The upper lane follows the assigned AMR, which first rides an elevator to the pickup floor if it is elsewhere; the lower lane follows the car that serves its loaded trip. Pickup and drop-off times include movement within a floor (Assumption A4). A request with the same origin and destination floors can join the trip while capacity remains, until loading ends at L_e . Durations are illustrative.

The warehouse management system selects, at the release epoch, a wave of fixed size n from a finite pool of K candidate waves. Resource quantities and operational rules are given, so the decision

concerns which orders enter the wave. The fixed size keeps candidates comparable and rules out shortening the completion time by releasing fewer orders. The order data and the candidate pool are known at the release epoch, whereas the representation of the elevator subsystem is uncertain and is described by a family of evaluators. The main formulation selects a structural class, defined by the floor distribution and directional balance of its candidate waves, and implements that choice through a fixed within-class selection rule. Formulating the decision over classes ties the release choice to interpretable properties of a wave. Because the classes are defined by quantiles within the pool, the same class definitions apply to any pool in which every class is nonempty. Class-level performance is obtained by evaluating the candidates of each class, exactly or from a sample, and is compared across elevator representations. Direct candidate selection is specific to one pool and provides a finite-pool benchmark, whose exact value is obtained by evaluating all candidates under the specified evaluator. The decision problem is to choose the class whose median wave makespan is smallest under the least favorable evaluator; the benchmark selects, for a given evaluator, the candidate whose makespan is smallest (Section 3.4).

The formulation rests on the following assumptions.

A1 (single isolated wave): Each decision instance concerns one wave released at time t_w . All AMRs and elevator resources are available on a common floor f^0 at t_w , with no outstanding reservations.

No other wave shares these resources, and the evaluation ends when the last selected order is delivered; backlog from earlier waves and interaction with later waves are outside the model.

A2 (single-load AMRs): An AMR carries one order at a time, and the AMR that collects an order also delivers it.

A3 (order information): Every available order has a known source floor, destination floor, and ready-time offset $r_o \geq 0$; the order becomes eligible for processing at $t_w + r_o$.

A4 (intra-floor operations): Movement and handling within a floor are represented by fixed pickup and drop-off service durations; intra-floor routing and congestion are not modeled.

A5 (elevator trips): A trip serves one origin-destination pair and consists of an empty repositioning leg from the floor at which the car last became available to the request origin, loading, a loaded leg from origin to destination, and unloading; the travel time of each leg is proportional to the number of floors traversed. An AMR rides an elevator whenever a movement crosses floors.

A6 (evaluator uncertainty): The elevator subsystem is represented by one of several evaluators (Section 3.5) that differ in their representation of capacity and trip-time variability. The applicable representation is not known at the release epoch, so decisions are evaluated under more than one representation; Section 3.4 specifies which representations enter the robust selection, and which one assesses a selected class under trip-time variability.

3.2 Notation and decision variables

3.2.1 Basic indices and parameters

Tables 3.1 and 3.2 summarize the basic indices and input parameters. Candidate-wave descriptors and operational state quantities are introduced with their definitions in Sections 3.3 and 3.5.

Table 3.1. Basic indices and sets

Symbol	Definition
$f \in \mathcal{F} = \{1, \dots, F\}$	warehouse-floor index

$o \in \mathcal{O}$	order index in the available order pool
$a \in \mathcal{A} = \{1, \dots, A\}$	AMR index
$e \in \mathcal{E} = \{1, \dots, E\}$	physical-elevator index
$k \in \mathcal{K} = \{1, \dots, K\}$	candidate-wave index
$q \in \mathcal{Q}$	selectable structural-class label (Section 3.3.3); the reference class, labeled $q = 0$, lies outside $\mathcal{Q}$ and serves only as a comparison baseline
$m \in \mathcal{M} = \{M_1, M_2, M_3\}$	elevator-evaluator index

Table 3.2. Input parameters

Symbol	Definition
F	number of warehouse floors
A	number of AMRs
E	number of physical elevator cars
c	maximum number of AMRs sharing one physical elevator trip
n	fixed number of orders in a wave
K	number of candidate waves in the finite pool
t_w	decision and release epoch of the wave
$f^0 \in \mathcal{F}$	common initial floor of the AMRs and elevator resources
$s_o, d_o \in \mathcal{F}$	source and destination floors of order o
$r_o \geq 0$	ready-time offset relative to t_w
$\gamma > 0$	elevator travel time per traversed floor
$\tau^L, \tau^U > 0$	elevator loading and unloading durations
$\tau^P, \tau^D > 0$	AMR pickup and drop-off service durations
$\alpha \in (0, 0.5)$	tail probability for structural-class construction
$\sigma > 0$	standard deviation of the logarithm of each phase multiplier under M_3

The resource counts, capacity, and candidate-pool size are positive integers, and $1 \leq n \leq |\mathcal{O}|$. All service durations and time offsets use the same time unit. The numerical values of the parameters in Table 3.2, the generator of the order pool $\mathcal{O}$ and of the ready-time offsets, and the sample sizes used to estimate class statistics are specified in Section 5.

3.2.2 Decision variables

The class formulation chooses a class label $q \in \mathcal{Q}$, and the candidate benchmark chooses a candidate index $k \in \mathcal{K}$; each decision selects one element of a finite set. Section 3.4 states the two decisions as optimization problems, and Section 3.5.1 writes both as one binary program in the variables of Table 3.3, with score coefficients supplied by the operational evaluation.

Table 3.3. Variables in the class and candidate formulations

Variable	Domain	Definition
w_s	$\{0, 1\}$	1 if option $s \in S$ is selected, and 0 otherwise ($S = \mathcal{Q}$ for the class formulation, $S = \mathcal{K}$ for the candidate benchmark)
η	$\mathbb{R}_+$	auxiliary upper bound on the selected option's scores over the evaluator family of the program

3.2.3 Evaluation notation

Each candidate wave is evaluated by reserving resources in a prescribed processing sequence. For candidate k , W^k is its order set and π^k is the processing sequence recorded when its orders are drawn during candidate generation (Section 3.3.1). The pair (W^k, π^k) is fixed before evaluation and shared by all evaluators. The index $j \in \mathcal{J}_n = \{1, \dots, n\}$ denotes a position in the sequence:

$$\pi^k = (o_1^k, \dots, o_n^k) \quad (1)$$

Under the generation procedure, the recorded sequence π^k is equally likely to be any permutation of the order set W^k . Because each candidate keeps its recorded sequence under every evaluator, differences between evaluators on the same candidate do not arise from resequencing. Across candidates, the sequences vary and contribute to the outcome variation within a class. All class and candidate results use the recorded sequences; Section 5 reports one alternative operational dispatch rule, which reorders the sequence, as a separately labeled robustness check.

Under M_3 , this evaluation also incorporates variation in trip-phase durations. Each newly reserved trip receives four multipliers $Z^{\text{rep}}, Z^{\text{load}}, Z^{\text{trav}}, Z^{\text{unload}}$ for empty repositioning, loading, loaded travel, and unloading, respectively. These multipliers are independent across phases and trips, and each multiplier Z satisfies

$$\log Z \sim \mathcal{N}\left(-\frac{\sigma^2}{2}, \sigma^2\right), \quad \mathbb{E}[Z] = 1 \quad (2)$$

The collection of phase multipliers used in a wave evaluation is denoted by $\xi \in \Xi$, where Ξ is the realization space. The phase multipliers are independent of the within-class candidate draw $\kappa \sim \nu_q$ introduced in Section 3.3.3. Given a candidate k , an evaluator m , and, when $m = M_3$, a phase-time realization ξ , the operational rules determine all state quantities and order completion times. The dependence of operational states on these inputs is omitted from their notation.

3.3 Candidate waves and structural classes

3.3.1 Candidate waves

Each candidate index $k \in \mathcal{K}$ identifies an order set $W^k \subseteq \mathcal{O}$ and its recorded processing sequence π^k . Each candidate is generated by drawing n distinct orders uniformly at random from $\mathcal{O}$, and the K candidates are generated independently before selection. Evaluators compared on the same decision, as in the robust selection of Section 3.4.2, use the same pool; Section 5 states whether the pools of each experiment are shared across evaluators. Every candidate has the fixed size

$$|W^k| = n, \quad k \in \mathcal{K} \quad (3)$$

3.3.2 Wave descriptors

The floor distribution of a wave is measured using both endpoints of every order. For a candidate wave W , $p_f(W)$ denotes the proportion of its $2|W|$ endpoints located on floor f :

$$p_f(W) = \frac{1}{2|W|} \sum_{o \in W} [\mathbb{I}(s_o = f) + \mathbb{I}(d_o = f)], \quad f \in \mathcal{F} \quad (4)$$

where $\mathbb{I}(\cdot)$ is the indicator function. Endpoint dispersion is represented by the entropy

$$C(W) = -\sum_{f \in \mathcal{F}} p_f(W) \log p_f(W), \quad 0 \log 0 = 0 \quad (5)$$

Here $\log$ denotes the natural logarithm. Larger C reflects greater diversity in the endpoint distribution across floor labels, arising from broader support or a more even distribution. C depends only on how endpoints are distributed over the floors, not on the distances between them; travel distances enter the evaluation through the floor differences in Equations (24), (28), and (30).

Directional imbalance measures the asymmetry between upward and downward order movements. The numbers of upward and downward orders are

$$N^\uparrow(W) = \sum_{o \in W} \mathbb{I}(d_o > s_o), \quad N^\downarrow(W) = \sum_{o \in W} \mathbb{I}(d_o < s_o). \quad (6)$$

The absolute difference is normalized by the number of cross-floor orders:

$$I(W) = \begin{cases} \frac{|N^\uparrow(W) - N^\downarrow(W)|}{N^\uparrow(W) + N^\downarrow(W)}, & N^\uparrow(W) + N^\downarrow(W) > 0, \\ 0, & N^\uparrow(W) + N^\downarrow(W) = 0. \end{cases} \quad (7)$$

Thus $I \in [0, 1]$, with zero indicating balanced cross-floor counts or the absence of cross-floor orders, and one indicating movement in only one direction. Same-floor orders do not contribute to either count. The ready-time dispersion T measures how order eligibility is spread within the wave. The mean ready-time offset is $\bar{r}_W = |W|^{-1} \sum_{o \in W} r_o$, and T is the coefficient of variation of the offsets:

$$T(W) = \begin{cases} \frac{\sqrt{|W|^{-1} \sum_{o \in W} (r_o - \bar{r}_W)^2}}{\bar{r}_W}, & \bar{r}_W > 0, \\ 0, & \bar{r}_W = 0. \end{cases} \quad (8)$$

Smaller T indicates more compact ready-time offsets relative to their mean. When all orders are ready at the release epoch, $T = 0$. The resulting descriptor vector is

$$\Phi(W) = (C(W), I(W), T(W)). \quad (9)$$

3.3.3 Structural classes and within-class selection

The class construction uses the lower and upper tails of C and I ; T records temporal variation separately and does not enter the corner classes or the selection problems of this section. It is retained in Equation (9) as a descriptor of the released wave under staggered ready times and is used only in the sensitivity analyses of Section 5. For descriptor $X \in \{C, I\}$, $Q_u(X)$ denotes its empirical u -quantile over the candidate pool, computed by linear interpolation between the adjacent order statistics at position $1 + (K-1)u$. A common tail probability α defines four candidate-index sets:

$$\begin{aligned} \mathcal{K}_{\text{HC}} &= \{k \in \mathcal{K} : C(W^k) \geq Q_{1-\alpha}(C)\} \\ \mathcal{K}_{\text{LC}} &= \{k \in \mathcal{K} : C(W^k) \leq Q_\alpha(C)\} \\ \mathcal{K}_{\text{HI}} &= \{k \in \mathcal{K} : I(W^k) \geq Q_{1-\alpha}(I)\} \\ \mathcal{K}_{\text{LI}} &= \{k \in \mathcal{K} : I(W^k) \leq Q_\alpha(I)\} \end{aligned} \quad (10)$$

A corner class consists of candidates lying in a tail region of both descriptors. In the labels HH, HL, LH, LL, the first letter records the tail of C and the second the tail of I . Section 5 writes

these labels as HC-HI, HC-LI, LC-HI, and LC-LI, respectively. The candidate-index supports are

$$\begin{aligned}\mathcal{K}_{HH} &= \mathcal{K}_{HC} \cap \mathcal{K}_{HI} \\ \mathcal{K}_{HL} &= \mathcal{K}_{HC} \cap \mathcal{K}_{LI} \\ \mathcal{K}_{LH} &= \mathcal{K}_{LC} \cap \mathcal{K}_{HI} \\ \mathcal{K}_{LL} &= \mathcal{K}_{LC} \cap \mathcal{K}_{LI}\end{aligned}\tag{11}$$

The selectable labels form $\mathcal{Q} = \{HH, HL, LH, LL\}$. A class-selection instance requires $\mathcal{K}_q \neq \emptyset$ for every $q \in \mathcal{Q}$, so that the class median in Equation (14) is defined for every selectable class. For continuous and independent descriptors, each class contains approximately a fraction α^2 of the pool. The fraction can differ when C and I are dependent. It can also differ when a descriptor takes few distinct values, because the inclusive thresholds in Equation (10) assign every candidate tied at a threshold to the tail set. The classes need not cover the pool, since candidates outside the four intersections belong to no class. Conversely, when ties make the lower and upper thresholds of a descriptor coincide, a candidate can belong to more than one class. Section 5 reports the realized class sizes and overlaps.

Each selectable class is implemented by uniform sampling over its candidate indices, with the fixed within-class distribution

$$v_q(k) = \begin{cases} \frac{1}{|\mathcal{K}_q|}, & k \in \mathcal{K}_q, \\ 0, & k \notin \mathcal{K}_q, \end{cases} \quad q \in \mathcal{Q}, k \in \mathcal{K}\tag{12}$$

Selecting class q releases the wave W^κ with $\kappa \sim v_q$. The class choice thus determines the distribution of the released wave, while the within-class rule is fixed. Repeated draws from v_q are independent draws with replacement from the finite set $\mathcal{K}_q$, so the same candidate can be released, or evaluated, more than once; the candidate identity of every evaluated draw is recorded, so that repeated candidates in an evaluated sample can be identified. The unrestricted pool defines a reference class, labeled $q = 0$, with $v_0(k) = 1/K$ for every $k \in \mathcal{K}$. It is not a decision option; it is the baseline against which Section 4 measures the value of class selection.

3.4 Objective functions

3.4.1 Wave completion measure

With the candidate, evaluator, and phase-time realization made explicit, $t_{jkm}^C(\xi)$ denotes the completion time of the order in position j of candidate k (Section 3.5.1). Wave makespan measures elapsed time from release until the last selected order is delivered:

$$Y_{km}(\xi) = C_{\max}(W^k, \pi^k, m, \xi) = \max_{j \in \mathcal{J}_n} t_{jkm}^C(\xi) - t_W\tag{13}$$

For M_1 and M_2 , the outcome is deterministic and the argument ξ is omitted. Under M_3 , Equation (13) defines the makespan for one phase-time realization.

3.4.2 Robust structural-class selection

The performance of a class is the median makespan induced by its fixed release distribution. We use the median because Section 4 states its class-level diagnosis for it and Section 5 reports it for each class, so the decision problem, the analysis, and the experiments share one estimand. When the median is non-unique, we use the midpoint of its median interval. For $q \in \mathcal{Q} \cup \{0\}$ and $m \in \mathcal{M}$, the class median is defined as

$$\mu_{qm}^{0.5} = \text{Med}_{\kappa \sim \nu_q, \xi} [Y_{\kappa m}(\xi)] \quad (14)$$

For M_1 and M_2 , variation in Equation (14) arises from the candidate draw. Under M_3 , the distribution includes both candidate variation and phase-time noise, and the median is taken over this joint outcome distribution. When evaluators are compared, the same draw $\kappa \sim \nu_q$ is evaluated under every evaluator, with the phase-time realization ξ drawn independently of κ . The corresponding class mean is $\bar{\mu}_{qm} = \mathbb{E}_{\kappa \sim \nu_q, \xi} [Y_{\kappa m}(\xi)]$. This mean enters the complementary distributional analysis of Section 4 and its numerical assessment in Section 5; the class-selection objective uses the median.

Under a single evaluator m , the nominal class selection is $q_m \in \arg \min_{q \in \mathcal{Q}} \mu_{qm}^{0.5}$. The deterministic evaluator family is $\mathcal{M}_D = \{M_1, M_2\}$. Its two members represent two hypotheses on how elevator capacity acts: as parallel single-request slots under M_1 or as shared trips under M_2 . The robust selection hedges against this structural uncertainty, and Section 4.3 gives sufficient conditions under which the two evaluators are ordered. M_3 retains the reservation rules of M_2 and adds mean-one phase multipliers (Equation (2)). It is therefore a perturbation of M_2 rather than a third capacity hypothesis, and it is used to assess a selected class under trip-time variability. The robust class decision minimizes the larger of its two median makespans:

$$q^* \in \arg \min_{q \in \mathcal{Q}} \max_{m \in \mathcal{M}_D} \mu_{qm}^{0.5} \quad (15)$$

The class medians are not observed at the release epoch; Section 4.1 obtains them by evaluating the candidates of each class, exactly or by sampling, and applies the selection rule to the resulting scores. Section 4 also diagnoses a descriptor-based class rule that selects a label $q_\Phi \in \mathcal{Q}$ from the signs of a relation between makespan and (C, I) fitted on an evaluated training sample under a given evaluator; that rule is distinct from the within-class draw of Equation (12), and its fitting sample is specified in Section 5.

3.4.3 Candidate-wave benchmarks

Direct candidate selection uses deterministic scores

$$\ell_{km} = C_{\max}(W^k, \pi^k; m), \quad k \in \mathcal{K}, m \in \mathcal{M}_D \quad (16)$$

For an evaluator $m \in \mathcal{M}_D$, the pool optimum $k_m \in \arg \min_{k \in \mathcal{K}} \ell_{km}$ attains the smallest score,

$$\ell_{k_m m} = \min_{k \in \mathcal{K}} \ell_{km} \quad (17)$$

Because every selection rule of Section 3.4 releases a wave from the same pool, Equation (17) is a lower bound on the makespan attainable under evaluator m by any such rule.

3.5 Constraints and their explanation

3.5.1 Constraint formulation

The three evaluators are serial reservation models, deterministic except for the phase multipliers of M_3 . Orders are reserved one at a time along the processing sequence, so the evaluators do not simulate concurrent execution, and the request times they generate need not increase along the sequence. Fig. 3.1(b) illustrates the reservation of one order. Section 5 cross-validates the evaluators against an event-driven simulation that uses the same physical specification but executes trips concurrently.

Processing sequence and AMR assignment. Each order's complete route is reserved before the next order is considered. After the first $j-1$ orders have been reserved, H_a^{j-1} denotes the next availability time of AMR a and G_a^{j-1} the floor at which it becomes available; initially, $H_a^0 = t_w$ and $G_a^0 = f^0$. For a finite index set $\mathcal{X}$ and real values u_x , $x \in \mathcal{X}$, $\text{lexargmin}_{x \in \mathcal{X}}(u_x, x)$ denotes the smallest index among those attaining $\min_x u_x$. This rule makes each selection in Equations (18), (23), and (27) unique. Order o_j^k is assigned to the earliest-available AMR, with ties resolved by AMR index, and begins at

$$a_j = \text{lexargmin}_{a \in \mathcal{A}}(H_a^{j-1}, a), \quad t_j^0 = \max\{H_{a_j}^{j-1}, t_w + r_{o_j^k}\} \quad (18)$$

An elevator request specifies an origin g , a destination $h \neq g$, and a request time t . The elevator reservation state is denoted by $\mathbf{S}_m$. The request updates this state and returns a completion time:

$$(\hat{t}, \mathbf{S}_m^+) = \mathcal{R}_m(\mathbf{S}_m, g, h, t; \xi) \quad (19)$$

Within a fixed evaluation, $\mathcal{R}_m(g, h, t)$ denotes the returned time $\hat{t}$, with every call replacing $\mathbf{S}_m$ by

$\mathbf{S}_m^+$. The assigned AMR first reaches the order's source and completes pickup:

$$t_j^S = \begin{cases} \mathcal{R}_m(G_{a_j}^{j-1}, s_{o_j^k}, t_j^0), & G_{a_j}^{j-1} \neq s_{o_j^k}, \\ t_j^0, & G_{a_j}^{j-1} = s_{o_j^k}, \end{cases} \quad t_j^P = t_j^S + \tau^P \quad (20)$$

It then reaches the destination and completes delivery:

$$t_j^D = \begin{cases} \mathcal{R}_m(s_{o_j^k}, d_{o_j^k}, t_j^P), & s_{o_j^k} \neq d_{o_j^k}, \\ t_j^P, & s_{o_j^k} = d_{o_j^k}, \end{cases} \quad t_j^C = t_j^D + \tau^D \quad (21)$$

After delivery, the assigned AMR becomes available at the destination:

$$H_{a_j}^j = t_j^C, \quad G_{a_j}^j = d_{o_j^k} \quad (22)$$

For every $a \neq a_j$, $H_a^j = H_a^{j-1}$ and $G_a^j = G_a^{j-1}$.

Throughput abstraction M_1 . Evaluator M_1 represents E elevators of capacity c by Ec independent

single-request service slots, indexed by $v \in \mathcal{V}_l = \{1, \dots, Ec\}$. Each slot has a next availability time B_v and a floor G_v associated with that availability, initially $B_v = t_w$ and $G_v = f^0$. A request uses

$$v^* = \underset{v \in \mathcal{V}_l}{\text{lexargmin}}(B_v, v) \quad (23)$$

Its completion time is

$$\mathcal{R}_{M_1}(g, h, t) = \max\{t, B_{v^*}\} + \gamma |G_{v^*} - g| + \tau^L + \gamma |g - h| + \tau^U \quad (24)$$

The selected slot is updated to $B_{v^*} \leftarrow \mathcal{R}_{M_1}(g, h, t)$ and $G_{v^*} \leftarrow h$; all other slots retain their states. The slots represent capacity as parallelism: E cars of capacity c under M_1 coincide with Ec cars of unit capacity, and every request occupies a slot for a full trip. M_1 and M_2 use the same trip durations; they differ only in the number of servers that can run in parallel and in whether requests share a trip. Each slot carries one request per trip, the single-rider assumption that Chakravarty et al. (2025) adopt for multi-robot lift scheduling, in which each lift serves one robot at a time.

True co-occupancy batching M_2 . Evaluator M_2 uses E physical cars, each accommodating at most c AMRs on a trip. For each car e , the evaluator stores its next availability B_e , associated floor G_e , and latest reserved trip. That trip is described by its origin-destination pair $(\bar{s}_e, \bar{d}_e)$, loading-end

time L_e , completion time D_e , and reserved occupancy P_e . Initially, $B_e = D_e = t_w$, $G_e = f^0$, no trip pair is stored, $L_e = -\infty$, and $P_e = 0$. A request can share a stored trip when its origin and destination match those of the trip, the trip has remaining capacity, and the request time is no later than the end of loading. The boardable-car set is

$$\mathcal{B}(g, h, t) = \{e \in \mathcal{E} : (\bar{s}_e, \bar{d}_e) = (g, h), P_e < c, t \leq L_e\} \quad (25)$$

If $\mathcal{B}(g, h, t) \neq \emptyset$, the lowest-index eligible car is selected:

$$e^* = \min \mathcal{B}(g, h, t), \quad \mathcal{R}_{M_2}(g, h, t) = D_{e^*}, \quad P_{e^*} \leftarrow P_{e^*} + 1 \quad (26)$$

The joining request shares the trip's stored completion time; apart from the occupancy update in Equation (26), the car's reservation record is unchanged. A request may also join a trip that starts after its request time, in which case it waits for that trip. When $\mathcal{B}(g, h, t) = \emptyset$, the request creates a new trip on the earliest-available car, with equal availability resolved by car index:

$$e^* = \underset{e \in \mathcal{E}}{\text{lexargmin}}(B_e, e) \quad (27)$$

Using the car's pre-update state, the new loading-end and completion times are

$$\begin{aligned} L_{e^*} &= \max\{t, B_{e^*}\} + \gamma |G_{e^*} - g| + \tau^L \\ D_{e^*} &= L_{e^*} + \gamma |g - h| + \tau^U \end{aligned} \quad (28)$$

The elevator returns $\mathcal{R}_{M_2}(g, h, t) = D_{e^*}$ and updates the car's reservation record:

$$B_{e^*} \leftarrow D_{e^*}, \quad G_{e^*} \leftarrow h, \quad (\bar{s}_{e^*}, \bar{d}_{e^*}) \leftarrow (g, h), \quad P_{e^*} \leftarrow 1 \quad (29)$$

The new trip replaces that car's stored trip record, and subsequent requests test Equation (25) against the latest record of each car. A stored trip serves one origin-destination pair and makes no intermediate stops; a request with a different pair, or one arriving after loading has ended, uses a separate trip. M_2 shares a trip only among requests with the same origin and destination. This is a restricted form of the destination-based grouping of destination group control, in which requests from a common origin are assigned to cars by destination until the car capacity is reached (So et al., 2022).

Stochastic batching M_3 . Evaluator M_3 retains the car-assignment, boarding, and capacity rules of M_2 . The loading-end and completion times of a new trip become

$$\begin{aligned} L_{e^*} &= \max\{t, B_{e^*}\} + \gamma |G_{e^*} - g| Z^{\text{rep}} + \tau^L Z^{\text{load}} \\ D_{e^*} &= L_{e^*} + \gamma |g - h| Z^{\text{trav}} + \tau^U Z^{\text{unload}} \end{aligned} \quad (30)$$

The reservation update follows Equation (29). Both a newly reserved request and a joining request return $\mathcal{R}_{M_3}(g, h, t) = D_{e^*}$. Requests joining the same trip share this stored completion time, with no additional phase draws.

Resource conditions. The reservation rules enforce the following conditions, which are the feasibility requirements of the wave-level schedule:

$$\begin{aligned} t_j^0 &\geq t_W + r_{o_j^i}, j \in \mathcal{J}_n, \\ t_{j'}^0 &\geq t_j^C, j < j', a_{j'} = a_j, \\ P_e &\leq c, e \in \mathcal{E} \text{ under } M_2 \text{ and } M_3 \end{aligned} \quad (31)$$

The first condition requires order readiness, the second requires that an AMR start its next order only after delivering the current one, and the third bounds the occupancy of a shared trip. In addition, trips reserved on the same car, or on the same slot under M_1 , do not overlap. Each new trip begins its empty repositioning at $\max\{t, B_e\}$, where B_e is the completion time of the previous trip on that car (B_e for a slot under M_1). These conditions are implied by Equations (18) to (30); the first two hold for all three evaluators, and the third for the two evaluators under which trips are shared.

Selection program. Both decisions of Section 3.4 select one option from a finite index set $\mathcal{S}$ with score coefficients c_{sm} : the class formulation has $\mathcal{S} = \mathcal{Q}$, $c_{sm} = \mu_{sm}^{0.5}$, and evaluator family $\mathcal{M}' = \mathcal{M}_D$; the pool optimum under evaluator m has $\mathcal{S} = \mathcal{K}$, $c_{sm} = \ell_{sm}$, and $\mathcal{M}' = \{m\}$. With the variables of Table 3.3, the common program is

$$\min_{w, \eta} \eta \quad \text{s.t.} \quad \begin{cases} \sum_{s \in \mathcal{S}} w_s = 1, \\ \eta \geq \sum_{s \in \mathcal{S}} c_{sm} w_s, m \in \mathcal{M}', \\ w_s \in \{0, 1\}, s \in \mathcal{S}, \\ \eta \geq 0 \end{cases} \quad (32)$$

Equation (32) reproduces Equation (15) for $\mathcal{M}' = \mathcal{M}_D$ and Equation (17) for $\mathcal{M}' = \{m\}$.

3.5.2 Explanation of constraints

Processing sequence and AMR assignment. The processing sequence is the candidate-specific permutation π^k in Equation (1) and is not reordered by readiness. An order that is not yet eligible holds its assigned AMR until $t_w + r_o$, which the maximum in Equation (18) enforces together with resource availability. Equations (20) and (21) activate an elevator request only when a movement crosses floors. A same-floor order can still require an initial elevator trip if its assigned AMR is on a different floor. Pickup and drop-off advance the AMR clock by their fixed service durations, and each order updates the state of its assigned AMR, so subsequent assignments use the resulting availability times.

Throughput abstraction M_1 . Equation (24) charges every request its waiting time and a complete trip of repositioning, loading, loaded travel, and unloading. Capacity therefore acts only through the number of slots that can serve requests at the same time, and two requests with the same origin and destination never share a trip.

True co-occupancy batching M_2 . Trips are reserved in the order of the processing sequence, so a new trip starts only after the trips already reserved on its car have been completed, even when its request time is earlier. The boarding test in Equation (25) also takes priority over new-trip assignment in Equations (27)–(29), so an eligible shared trip is selected even when a dedicated trip on another car could finish sooner.

Stochastic batching M_3 . Only the four elevator trip phases carry multipliers. Waiting times carry none and change only as a consequence of the perturbed durations of earlier trips, while pickup and drop-off durations remain fixed.

Resource conditions. Equation (31) collects the feasibility requirements that the reservation rules already enforce: order readiness through the maximum in Equation (18), sequential use of each AMR through the availability update in Equation (22), and trip occupancy through the boarding test in Equation (25). They are stated separately so that the wave-level schedule can be checked without re-deriving the recursions.

Selection program. The first constraint of Equation (32) selects exactly one option; the score constraints bound η below by the selected option's score under each evaluator in $\mathcal{M}'$, and minimization makes η equal to the largest of those scores. A selected class is implemented through Equation (12); the pool optimum releases the selected candidate directly.